

Course Module: Sequence Models

Sara Latreche

Contents

1	Introduction to Recurrent Neural Networks	4
2	Sequence Models and Sequence Data	6
2.1	Motivating Example: Recipe Instruction Prediction	7
3	Hands-on Lab: Sequence Modeling with Recipe Instructions	8
3.1	Why Recurrent Neural Networks?	9
4	Different Types of RNNs	15
5	Language Models and Sampling	17
5.1	Training a Language Model	17
5.2	Sampling from a Language Model	18
5.3	Word-level vs. Character-level Models	18
6	Limitations of RNNs: The Vanishing Gradient Problem	20
7	Long Short-Term Memory (LSTM) Networks	22
8	Gated Recurrent Units (GRU)	24
9	Bidirectional and Deep RNNs	25
9.1	Bidirectional RNNs (BRNNs)	25
9.2	Deep RNNs	26

10 Word Embeddings	27
10.1 Introduction to Word Representations	27
10.2 The Distributional Hypothesis	29
10.3 Word2Vec Training Strategies	30
10.4 Interpretability and Latent Attributes	31
10.5 Visualizing High-Dimensional Embeddings	33
10.6 Static vs. Contextual Embeddings (Transformers)	33
11 Machine Translation	33
11.1 Conditional Language Models	33
11.2 Greedy Search vs. Beam Search	34
11.3 Refinements to Beam Search	34
11.4 Error Analysis in Beam Search	35
11.5 BLEU Score	35
12 Attention Models: Intuition and Implementation	36
12.1 Motivation	36
12.2 Intuitive Example: Date Normalization	37
12.3 Formal Description of Attention	37
12.4 Computing Attention Weights	38
12.5 Decoding with Attention	38
12.6 Advantages and Applications	39
13 Numerical Example: Attention Mechanism	39
14 Introduction to Transformers	41
14.1 Key Components of a Transformer	42
14.2 Self-Attention Intuition	42
14.3 Advantages of Transformers	42
14.4 From Attention to Transformers	43

15 Large Language Models (LLMs)	43
15.1 Key Characteristics of LLMs	43
15.2 Examples of LLMs	43
15.3 Why LLMs Work Well	44
15.4 Applications of LLMs	44
15.5 Inside CHAT GPT 2	45
15.5.1 1. The Embedding Layer: Mapping Text to Vector Space	45
15.5.2 The Transformer Block: The Engine of Inference	47
15.5.3 Output Probabilities	49
References	51

1 Introduction to Recurrent Neural Networks

The idea of modeling sequences—predicting the next word in a sentence, the next note in a melody, or the next event in a timeline—predates modern deep learning. Long before today’s powerful neural architectures, researchers were already fascinated with the possibility of teaching machines to remember and predict across time.

One of the earliest inspirations came from Markov chains in the early 20th century. Introduced by Andrey Markov in 1906 [1], these models showed that complex sequences, like letters in a poem, could be understood by analyzing the probability of each symbol given its predecessor. This was the first mathematical attempt to capture the structure of sequential data.

By the 1980s, the search for memory in artificial intelligence led to neural approaches. John Hopfield introduced the Hopfield network in 1982 [2], a model of associative memory. It could store patterns and retrieve them dynamically, inspiring the idea that networks could have an internal state that evolves over time. Although Hopfield networks were limited, they laid the groundwork for what would later become recurrent architectures.

The real leap came in 1986, when David Rumelhart, Geoffrey Hinton, and Ronald Williams published their landmark paper “Learning representations by back-propagating errors” [3], introducing backpropagation through time (BPTT). This allowed neural networks to be trained on sequential data, giving rise to the modern Recurrent Neural Network (RNN).

An RNN processes information one step at a time, updating a hidden state that acts as its memory. Unlike Markov chains, which only look at short context windows, RNNs can, in principle, capture long-term dependencies by repeatedly applying the same transformation as data flows through time.

Yet, early RNNs faced the vanishing and exploding gradient problem—making it extremely difficult to learn long-range dependencies. The breakthrough came in 1997, when Sepp Hochreiter and Jürgen Schmidhuber introduced the Long Short-Term Memory (LSTM) network [4]. With its gating mechanisms, LSTM overcame the memory bottleneck and became the backbone of sequence learning for two decades. Later, in 2014, the Gated Recurrent Unit (GRU), introduced by Kyunghyun Cho et al. [5], provided a simpler alternative with similar performance.

From speech recognition to machine translation, from time-series forecasting to music generation, RNNs became the foundation of sequence modeling before the rise of attention and transformers. They represented the first time machines could learn from history and make predictions about the future.

As Jürgen Schmidhuber later remarked [6], recurrent networks embody a simple yet profound idea:

Note

“The future can be predicted from the past—if you know how to remember.”

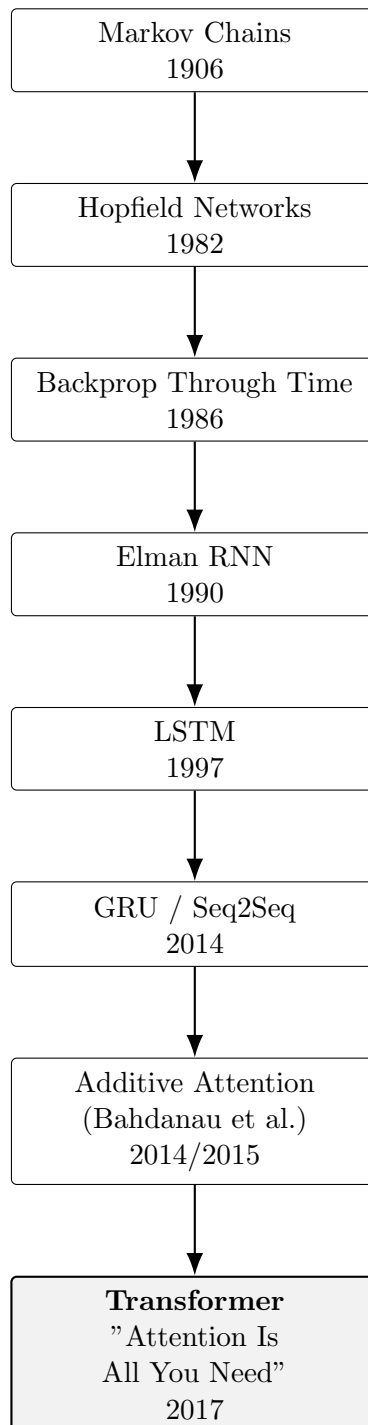


Figure 1: Historical evolution of sequence modeling: from Markovian chains to the Transformer.

2 Sequence Models and Sequence Data

Definition

Sequence Models are a class of models designed to process and make predictions on **ordered data**, where the order of elements matters. They are particularly useful when the current output depends not only on the current input but also on previous elements in the sequence.

Sequence Data refers to data where each observation is part of an ordered series, such as:

- Words in a sentence (text)
- Audio samples in speech or music
- DNA bases in genomics
- Video frames in an activity sequence
- Stock prices or sensor readings in time series

Sequence models include recurrent neural networks (RNNs), long short-term memory networks (LSTMs), gated recurrent units (GRUs), and attention-based models like Transformers. They are widely used in tasks such as speech recognition, music generation, machine translation, sentiment classification, video activity recognition, and named entity recognition.

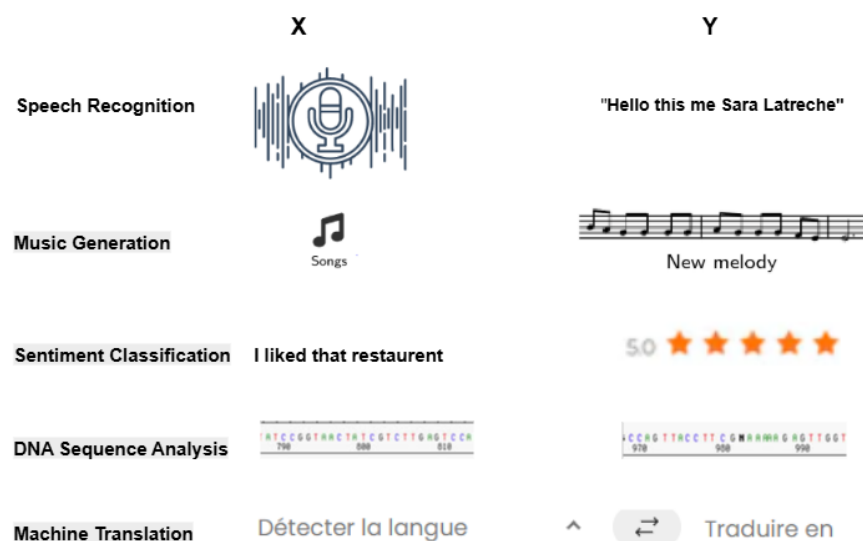


Figure 2: Illustrative examples of sequence data across multiple domains: text, audio, music, DNA, video, and other time series. Sequence models can learn patterns and dependencies in such data to make predictions.

2.1 Motivating Example: Recipe Instruction Prediction

Example

Recipe Sequence Modeling To understand sequence models, let's look at a motivating example from cooking. Consider the short recipe:

“Chop the onions, heat the pan, add oil, sauté onions, add spices, cook for 5 minutes, serve.”

Each step of the recipe can be represented as a sequence element for a model.

Input sequence: $X = \{X^{(1)}, \dots, X^{(7)}\}$

$X^{(1)} = \text{“Chop the onions”}, \quad X^{(2)} = \text{“Heat the pan”}, \quad \dots, \quad X^{(7)} = \text{“Serve”}.$

Output sequence: $Y = \{Y^{(1)}, \dots, Y^{(7)}\}$ represents the prediction at each step (the next instruction).

Sequence length: $T_x = 7$ in this example. Different recipes in a dataset may have variable lengths $T_x^{(i)}$.

Encoding Instructions for the Model

To feed the sequence into a model (RNN, LSTM, GRU), we need to represent each instruction numerically:

- Build a **vocabulary** of unique words across all recipes. Suppose the vocabulary size is 5,000.
- Convert each instruction $X^{(t)}$ into a **one-hot vector** of length 5,000:
 - “Chop” might correspond to position 1234 $\Rightarrow$ 1 at index 1234
 - “Onions” might correspond to position 245 $\Rightarrow$ 1 at index 245
- If the model encounters a word not in the vocabulary, use a special token <UNK>.

Note

Prediction with Sequence Models Once encoded, the sequence model can learn to predict the next instruction:

$$Y^{(t)} = f(X^{(1)}, \dots, X^{(t)})$$

where f is modeled by an RNN, LSTM, or GRU. Each step prediction depends on all previous steps.

Active Recall – Key Concepts

- Input sequence: $X = \{X^{(1)}, \dots, X^{(T_x)}\}$
- Output sequence: $Y = \{Y^{(1)}, \dots, Y^{(T_y)}\}$
- Variable-length sequences across examples
- Mapping from discrete word representations to predictions

Note: This same structure applies to other domains such as text, audio, DNA sequences, and video frames.

3 Hands-on Lab: Sequence Modeling with Recipe Instructions

NLTK Toolbox Introduction

NLTK (Natural Language Toolkit) is a Python library that provides tools for processing and analyzing text. In this lab, we will use it to:

- **Tokenize text:** Split text into words using `word_tokenize`.
- **Build vocabulary:** Extract all unique words and assign an index.
- **Handle unknown words:** Use a special token `<UNK>` if a word is not in the vocabulary.

Hands-on Lab

Recipe Instruction Prediction Lab We will use the following short recipe as our motivating example:

“Chop the onions, heat the pan, add oil, sauté onions, add spices, cook for 5 minutes, serve.”

Each step of the recipe will be treated as a sequence element. Your tasks are as follows:

1. **Step Tokenization:** Split the recipe into individual steps using the comma as a delimiter. Print all steps.
2. **Word Tokenization:** Tokenize each step into words using `word_tokenize`. Print the tokenized steps.
3. **Vocabulary Building:** Extract all unique words across all steps. Map each word to a unique index.
4. **One-hot Encoding:** Convert each step into a multi-hot vector where each word in the step is represented by a 1 in the vector. Print the shape of the resulting array.
5. **Prepare Input-Output Pairs:** Create input-output pairs to predict the next instruction given all previous steps. Print the shapes of X and Y.
6. **Optional RNN Prediction:** Using PyTorch, feed the input sequence to a simple RNN and print the output shape. No training is required.

Hint: For steps containing words not in your vocabulary, use a special token `<UNK>`.

3.1 Why Recurrent Neural Networks?

In our recipe example, we want to predict the **next step** given previous steps. One might think: why not just use a standard neural network (ANN)?

Problem 1 – Variable sequence lengths

- Recipes can have different numbers of steps.
- A standard ANN expects fixed-size inputs and outputs.
- Padding sequences to the same length is possible, but it doesn't fully capture the sequential nature of the data.

Problem 2 – Sharing features across positions

- A standard ANN learns features specific to fixed positions.
- If the word “add” is important in the first step, the ANN may not generalize to “add” appearing later.
- Ideally, features should **generalize across all positions**, similar to CNNs in images.

Problem 3 – Large number of parameters

- Each step is a one-hot vector of length equal to the vocabulary (e.g., 5,000).
- Flattening all steps into a single input vector creates enormous weight matrices.
- This is inefficient and prone to overfitting.

Definition

Recurrent Neural Networks (RNNs) **Recurrent Neural Networks (RNNs)** are designed to handle sequential data by maintaining a **hidden state** $a^{(t)}$ that carries information from previous steps.

Example

RNN Step-by-Step Intuition with Recipe Example Consider the recipe steps:

“Chop the onions, heat the pan, add oil, sauté onions, add spices, cook for 5 minutes, serve.”

- Time-step 1: $X^{(1)}$ = “Chop the onions” is fed into the network. Hidden state $a^{(1)}$ is computed and output $\hat{Y}^{(1)}$ predicts the next step (e.g., “Heat the pan”).
- Time-step 2: $X^{(2)}$ = “Heat the pan”, together with previous hidden $a^{(1)}$, produces $a^{(2)}$ and $\hat{Y}^{(2)}$ (predicting “Add oil”).
- Time-step 3: $X^{(3)}$ = “Add oil”, together with $a^{(2)}$, produces $a^{(3)}$ and $\hat{Y}^{(3)}$ (predicting “Sauté onions”).
- Continue this process until the final step $X^{(T_x)} = \text{“Serve”}$, which produces $a^{(T_x)}$ and predicts the “next step” if applicable.

Notation Summary:

- $X^{(t)}$: Input at time-step t (current recipe step)
- $a^{(t)}$: Hidden state carrying information from previous steps
- $\hat{Y}^{(t)}$: Output predicting the next step

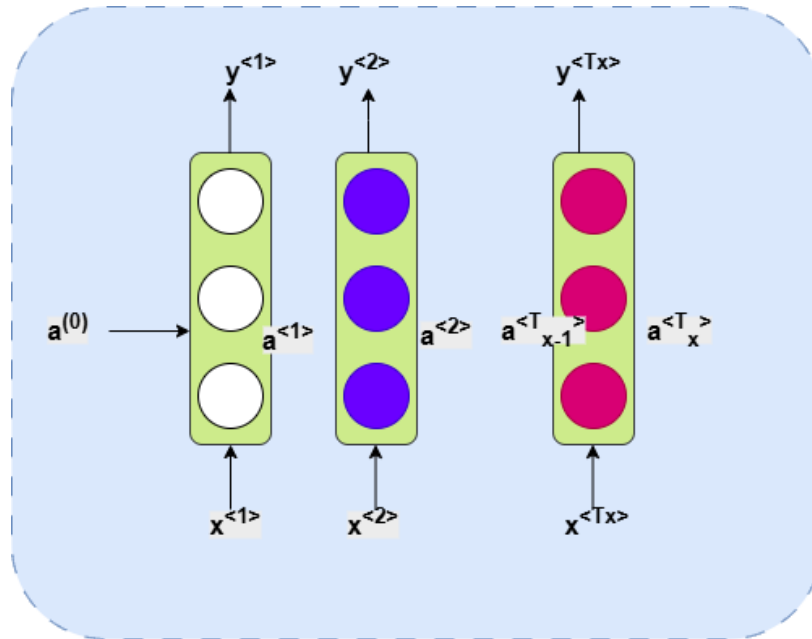


Figure 3: RNN Architecture: inputs $X^{(t)}$, hidden states $a^{(t)}$, outputs $\hat{Y}^{(t)}$. The hidden states carry information forward through time.

Key Advantages of RNNs

1. Handle **variable-length sequences** naturally.
2. **Shared parameters** across time steps reduce model size.
3. **Memory of previous steps** helps predict the next step in a sequence.

Note

Limitation Standard RNNs only use past information. To incorporate future information, we need **Bidirectional RNNs (BRNNs)**, which we will explore later.

Hint / Visualization

Imagine “unrolling” the RNN in time:

$$X^{(1)} \rightarrow [\text{Hidden}] \rightarrow \hat{Y}^{(1)} \quad \downarrow \quad X^{(2)} \rightarrow [\text{Hidden}] \rightarrow \hat{Y}^{(2)} \quad \downarrow \quad X^{(3)} \rightarrow [\text{Hidden}] \rightarrow \hat{Y}^{(3)} \quad \vdots$$

At each step, the hidden state passes information forward. The parameters W_{xh}, W_{hh}, W_{hy} are the same for all steps.

This is why RNNs are the natural choice for **sequential data** like recipes, text, audio, or DNA sequences.

Definition

Forward Propagation in RNNs Forward propagation in a recurrent neural network (RNN) computes the hidden states $a^{(t)}$ and outputs $\hat{Y}^{(t)}$ step by step.

Equations:

$$a^{(t)} = g(W_{aa}a^{(t-1)} + W_{ax}X^{(t)} + b_a)$$

$$\hat{Y}^{(t)} = g_y(W_{ya}a^{(t)} + b_y)$$

Where:

- $X^{(t)}$ is the input at time-step t .
- $a^{(t)}$ is the hidden state at time-step t , carrying information from previous steps.
- $\hat{Y}^{(t)}$ is the predicted output at time-step t .
- g and g_y are activation functions (e.g., tanh for hidden, sigmoid/softmax for output).

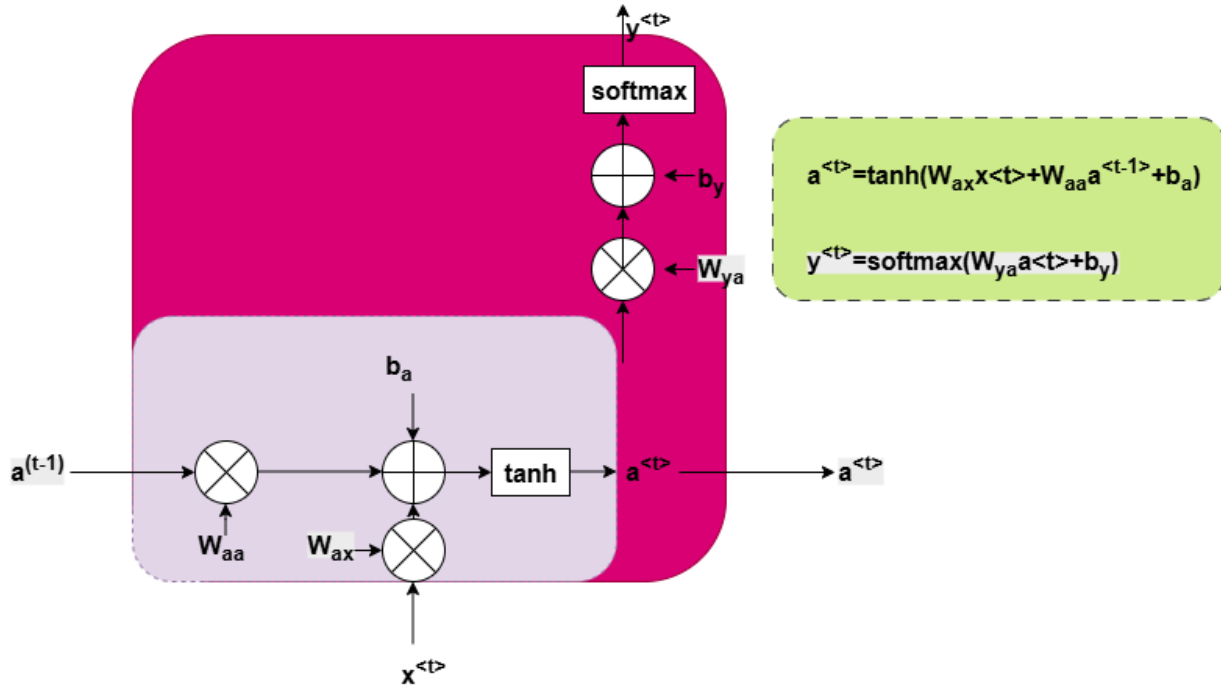


Figure 4: Single RNN Cell: computes $a^{<t>}$ and $\hat{Y}^{<t>}$ from $X^{<t>}$ and $a^{<t-1>}$.

Example

Forward Propagation Step-by-Step

- Start with $a^{<0>} = 0$ (zero vector initialization).
- Compute $a^{<1>} = g(W_{aa}a^{<0>} + W_{ax}X^{<1>} + b_a)$.
- Compute $\hat{Y}^{<1>} = g_y(W_{ya}a^{<1>} + b_y)$.
- Repeat for $t = 2$ to T_x : $a^{<t>} = g(W_{aa}a^{<t-1>} + W_{ax}X^{<t>} + b_a)$ and $\hat{Y}^{<t>} = g_y(W_{ya}a^{<t>} + b_y)$.

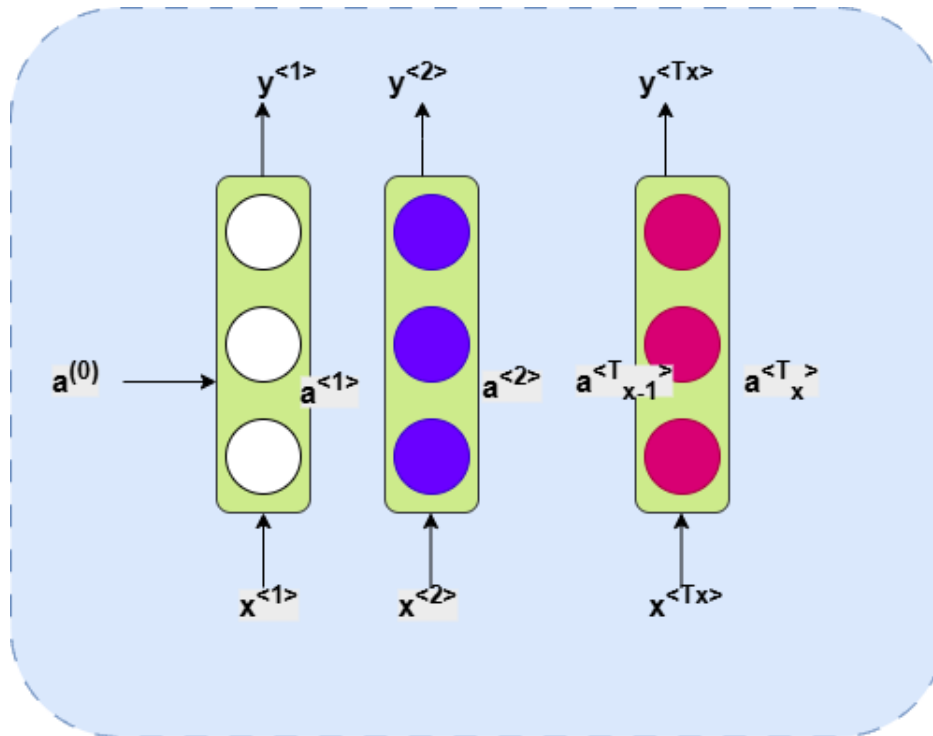


Figure 5: Unrolled RNN Architecture: forward propagation from $X^{(1)}$ to $X^{(Tx)}$. Hidden states $a^{(t)}$ carry memory through time.

Definition

Backward Propagation Through Time (BPTT) Backpropagation in RNNs is called **Backpropagation Through Time (BPTT)**. It computes gradients of the loss with respect to the parameters by moving backward through the unrolled network.

Steps:

- Compute the loss at each time-step t :

$$\mathcal{L}^{(t)} = -y^{(t)} \log \hat{Y}^{(t)} - (1 - y^{(t)}) \log(1 - \hat{Y}^{(t)})$$

- Compute total loss over sequence: $\mathcal{L} = \sum_{t=1}^{T_y} \mathcal{L}^{(t)}$.
- Propagate gradients **from right to left** in time:

$$\frac{\partial \mathcal{L}}{\partial W_{aa}}, \frac{\partial \mathcal{L}}{\partial W_{ax}}, \frac{\partial \mathcal{L}}{\partial W_{ya}}, \frac{\partial \mathcal{L}}{\partial b_a}, \frac{\partial \mathcal{L}}{\partial b_y}$$

- Update parameters using gradient descent.

Key intuition: Forward propagation “reads” the sequence left to right, carrying hidden states $a^{(t)}$. BPTT propagates gradients backward through these states, allowing the network to learn dependencies across time.

4 Different Types of RNNs

So far, we have studied the basic RNN architecture where the number of inputs T_x is equal to the number of outputs T_y . In practice, T_x and T_y may differ depending on the task. This gives rise to several types of RNN architectures.

Definition

Types of RNN Architectures:

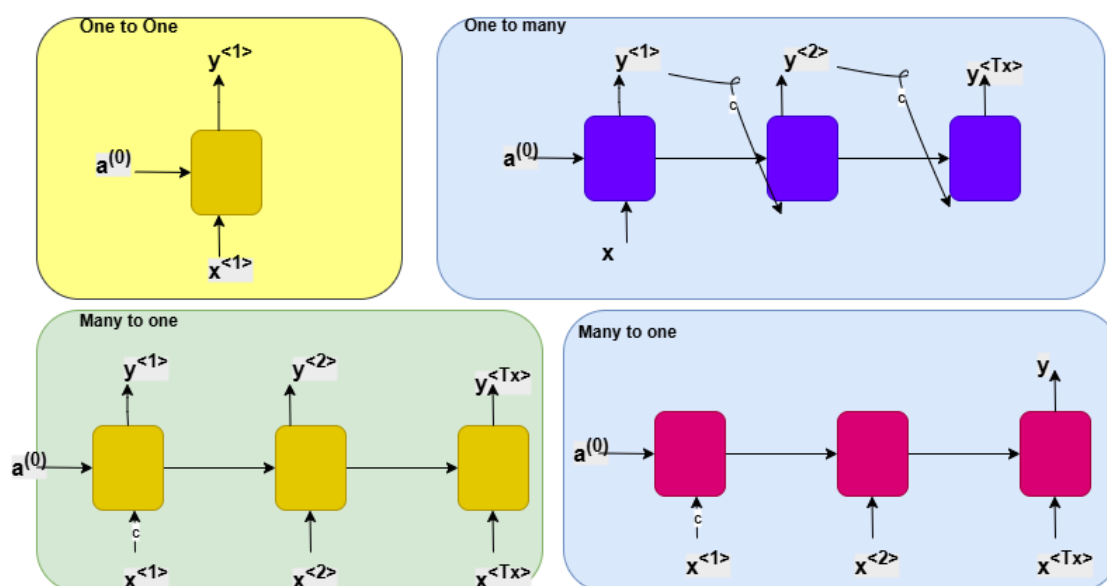
- **One-to-One:** Standard neural network setup. One input maps directly to one output (e.g., image classification).
- **One-to-Many:** A single input produces a sequence of outputs (e.g., image captioning: one image $\rightarrow$ multiple words).
- **Many-to-One:** A sequence of inputs produces a single output (e.g., sentiment analysis: a review $\rightarrow$ positive/negative).
- **Many-to-Many (Equal Length):** Input and output are both sequences of the same length (e.g., named entity recognition, part-of-speech tagging).
- **Many-to-Many (Different Lengths):** Input and output are sequences of different lengths (e.g., machine translation: French sentence $\rightarrow$ English sentence).

Note

Each of these architectures addresses a different class of problems. For example:

- Movie review classification is a **many-to-one** task.
- Captioning a photo is a **one-to-many** task.
- Translating French to English is a **many-to-many (different lengths)** task.

The figure below summarizes all of these cases in a single diagram.



5 Language Models and Sampling

A **language model (LM)** assigns a probability to a sequence of words. Formally, given a sentence $y^{(1)}, y^{(2)}, \dots, y^{(T_y)}$, the model estimates:

$$P(y^{(1)}, y^{(2)}, \dots, y^{(T_y)}) = \prod_{t=1}^{T_y} P(y^{(t)} \mid y^{(1)}, \dots, y^{(t-1)})$$

This means the probability of an entire sentence is obtained by multiplying the probabilities of each word, given the words that came before it.

Definition

What does a language model do? It tells us how likely a sentence is in a given language.

- In speech recognition, among multiple transcriptions, the LM picks the one that “sounds” most natural.
- In machine translation, it ensures that the translated output is a valid and fluent sentence.

5.1 Training a Language Model

To build a language model with an RNN:

1. Collect a large **corpus** of text (e.g., English sentences).
2. Tokenize sentences into words (or characters), and build a vocabulary.
3. Add a special token <EOS> (End of Sentence) to mark sentence boundaries.
4. Train the RNN to predict the next word given the previous ones:

$$\hat{y}^{(t)} = \text{Softmax}(W_y a^{(t)} + b_y)$$

where $a^{(t)}$ is the hidden state at time t .

Example: For the sentence *‘‘Cats average 15 hours of sleep a day’’*, the model learns to predict each word step by step:

$$P(\text{‘‘average’’} \mid \text{‘‘Cats’’}), \quad P(\text{‘‘15’’} \mid \text{‘‘Cats average’’}), \quad \dots$$

5.2 Sampling from a Language Model

Once trained, we can **generate new text** by sampling words sequentially:

1. Start with $a^{(0)} = 0$, $x^{(1)} = 0$.
2. Use the RNN to produce a probability distribution over the vocabulary.
3. Randomly sample the first word according to this distribution.
4. Feed this sampled word back into the RNN as input for the next timestep.
5. Repeat until the model outputs the **<EOS>** token (or a fixed length).

Example

Example of Sampling Suppose the model starts with predicting the first word. It may assign:

$$P(\text{"Cats"}) = 0.3, \quad P(\text{"The"}) = 0.25, \quad P(\text{"A"}) = 0.15, \quad \dots$$

If "Cats" is sampled, then at the next step the RNN predicts the distribution for the second word, conditioned on "Cats". This process continues until **<EOS>** is generated.

5.3 Word-level vs. Character-level Models

- **Word-level models:** Vocabulary consists of words. More natural, but limited by vocabulary size (need UNK for unknown words).
- **Character-level models:** Vocabulary consists of characters. No unknown words, but sequences are much longer and harder to train.

In summary, a language model predicts the probability of sentences and can generate new ones by sampling step by step. This makes it a core building block for speech recognition, translation, and many other NLP tasks.

Example

Practical Example: Language Model Sampling Consider a very small vocabulary:

$$V = \{\text{cats, sleep, play, a lot, EOS}\}$$

where **EOS** means “End of Sentence.”

Suppose a trained RNN language model learns the following probabilities:

Condition	Next Word Probabilities
Start	cats (0.6), sleep (0.2), play (0.1), a lot (0.05), EOS (0.05)
Previous word = cats	sleep (0.5), play (0.3), a lot (0.1), EOS (0.1)
Previous word = sleep	a lot (0.7), EOS (0.3)
Previous word = a lot	EOS (1.0)

Sampling process:

1. At the start, the model samples **cats**.
2. Given “cats”, it samples **sleep**.
3. Given “sleep”, it samples **a lot**.
4. Given “a lot”, it samples **EOS**, and the sentence ends.

Generated sentence: “cats sleep a lot”

Notice that if we repeat the process, we could get different outputs such as:

“cats play”, “sleep a lot”, “cats sleep EOS”

This illustrates how a language model can generate **novel sequences** while still respecting the statistical patterns it has learned.

Active Recall – Language Models

Try to recall the answers without looking back at your notes.

1. What is the **main task** of a language model?
2. In the context of a language model, what does the notation

$$P(y^{(1)}, y^{(2)}, \dots, y^{(T_y)})$$

represent?

3. Why is the **End of Sentence (EOS)** token useful when training an RNN-based language model?
4. What is the difference between a **word-level** and a **character-level** language model? Give one advantage and one disadvantage for each.
5. Suppose your vocabulary has the words {cats, sleep, play, EOS}. If your RNN predicts:

$$P(\text{cats}) = 0.6, \quad P(\text{sleep}) = 0.2, \quad P(\text{play}) = 0.1, \quad P(\text{EOS}) = 0.1$$

Which word is most likely to be sampled first? Could a different word still appear in the generated sentence? Why?

6. In language model sampling, how does the model decide on the **next input** for step $t + 1$?
7. How can you compute the probability of an entire sentence, e.g. “cats sleep a lot”?

6 Limitations of RNNs: The Vanishing Gradient Problem

Recurrent Neural Networks (RNNs) are powerful for modeling sequences, but they suffer from a major limitation: the **vanishing gradient problem**. This makes it hard for them to capture **long-term dependencies** in sequences such as natural language.

A Language Modeling Example

Consider the sentence:

“The cat, which already ate a bunch of food that was delicious . . . , was full.”

To predict “**was**”, the network must remember that the subject “*cat*” is singular. Now compare with:

“The cats, which already ate a bunch of food that was delicious . . . , were full.”

Here, the verb should be “**were**” because the subject is plural.

In both cases, the verb depends on information that appeared much earlier in the sentence. This requires the RNN to preserve information across many time steps.

Why RNNs Struggle with Long-Term Dependencies

Training an RNN involves two steps:

1. **Forward propagation:** passing information from left to right through the sequence.
2. **Backpropagation through time (BPTT):** propagating gradients from right to left to update earlier computations.

When the sequence is long (hundreds or thousands of time steps), the gradients used to update parameters:

$$\frac{\partial \mathcal{L}}{\partial W}$$

either:

- **Shrink exponentially** → **vanishing gradients**, meaning early inputs have almost no influence on later predictions.
- **Grow exponentially** → **exploding gradients**, meaning parameters can blow up and cause numerical errors (NaNs).

Note

In practice, vanishing gradients are the bigger problem for language models, because it prevents the network from learning long-term dependencies. Exploding gradients, while rarer, are catastrophic but can be mitigated with **gradient clipping**.

Connection to Deep Neural Networks

An RNN unrolled over T_x time steps is essentially a very deep neural network with T_x layers. Just as very deep networks suffer from gradient issues, so do RNNs:

Long sequence (e.g., $T_x = 1000$) $\Rightarrow$ “1000-layer network”.

This explains why RNNs often capture only **short-term context**, where outputs are influenced mainly by nearby inputs.

Definition

Gradient Clipping: A technique to prevent exploding gradients by rescaling gradient vectors when their norm exceeds a threshold.

Summary

- RNNs often fail at long-term dependencies due to the **vanishing gradient problem**.
- Exploding gradients are easier to handle using gradient clipping.
- Vanishing gradients remain a challenge — we will soon see architectures like **GRUs** and **LSTMs** that address this limitation.

Active Recall – Vanishing Gradients

1. In the cat/cats example, why is long-term memory required?
2. What is the main difference between vanishing and exploding gradients?
3. Why is gradient clipping a good solution for exploding but not vanishing gradients?
4. Why can an RNN unrolled over 1000 time steps be compared to a 1000-layer network?

7 Long Short-Term Memory (LSTM) Networks

Definition

Long Short-Term Memory (LSTM) networks are a type of recurrent neural network specifically designed to overcome the **vanishing gradient problem**. They do this by introducing a more sophisticated hidden unit structure that allows the network to **retain long-term dependencies**.

Example

Intuition Behind LSTMs Instead of passing information only through hidden states, LSTMs introduce a **cell state** $c^{(t)}$ that acts like a conveyor belt. This state flows almost unchanged through the sequence, with the network learning what information to **keep**, **update**, or **forget**.

The key idea is that the LSTM cell uses **gates**:

- **Forget gate:** Decides what information to discard.

$$f^{(t)} = \sigma(W_f[a^{(t-1)}, x^{(t)}] + b_f)$$

- **Input gate:** Decides what new information to store.

$$i^{(t)} = \sigma(W_i[a^{(t-1)}, x^{(t)}] + b_i)$$

$$\tilde{c}^{(t)} = \tanh(W_c[a^{(t-1)}, x^{(t)}] + b_c)$$

- **Cell state update:** Combine old and new information.

$$c^{(t)} = f^{(t)} \odot c^{(t-1)} + i^{(t)} \odot \tilde{c}^{(t)}$$

- **Output gate:** Controls what part of the cell state to output.

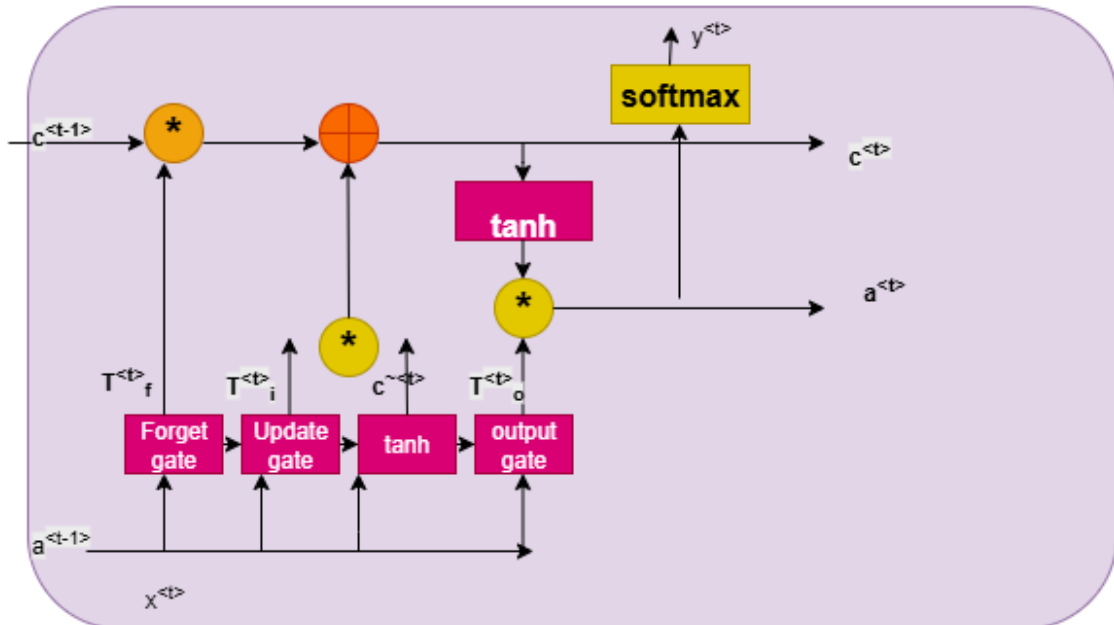
$$o^{(t)} = \sigma(W_o[a^{(t-1)}, x^{(t)}] + b_o)$$

$$a^{(t)} = o^{(t)} \odot \tanh(c^{(t)})$$

Note

- The **cell state** $c^{(t)}$ preserves long-term information with minimal modifications at each step.
- The **gates** learn what to remember and what to forget, allowing LSTMs to capture dependencies across long sequences.

The figure below illustrates the LSTM cell structure.



8 Gated Recurrent Units (GRU)

Definition

Gated Recurrent Units (GRU) are an improved variant of RNNs designed to handle long-range dependencies and mitigate the vanishing gradient problem. They do so by introducing gating mechanisms that control what information to keep, update, or discard over time.

Example

Intuition Behind GRUs Recall that in a basic RNN, the hidden state $a^{(t)}$ is updated using the previous state $a^{(t-1)}$ and the current input $x^{(t)}$. This structure often struggles to remember information across many time steps.

GRUs improve on this by introducing:

- An **update gate** $z^{(t)}$ that decides how much of the past information to keep.
- A **reset gate** $r^{(t)}$ that decides how much of the past should be forgotten when creating a candidate memory.

Think of the update gate as a switch: - If $z^{(t)} \approx 1$, the unit updates its memory with the new candidate. - If $z^{(t)} \approx 0$, the unit keeps its old memory almost unchanged. This makes GRUs very effective at deciding *when* to update information (e.g., when you encounter “the cat” in a sentence) and *when* to simply carry old information forward (e.g., keeping the subject as singular until the verb appears).

GRU equations:

$$z^{(t)} = \sigma(W_z[a^{(t-1)}, x^{(t)}] + b_z) \quad (\text{update gate})$$

$$r^{(t)} = \sigma(W_r[a^{(t-1)}, x^{(t)}] + b_r) \quad (\text{reset gate})$$

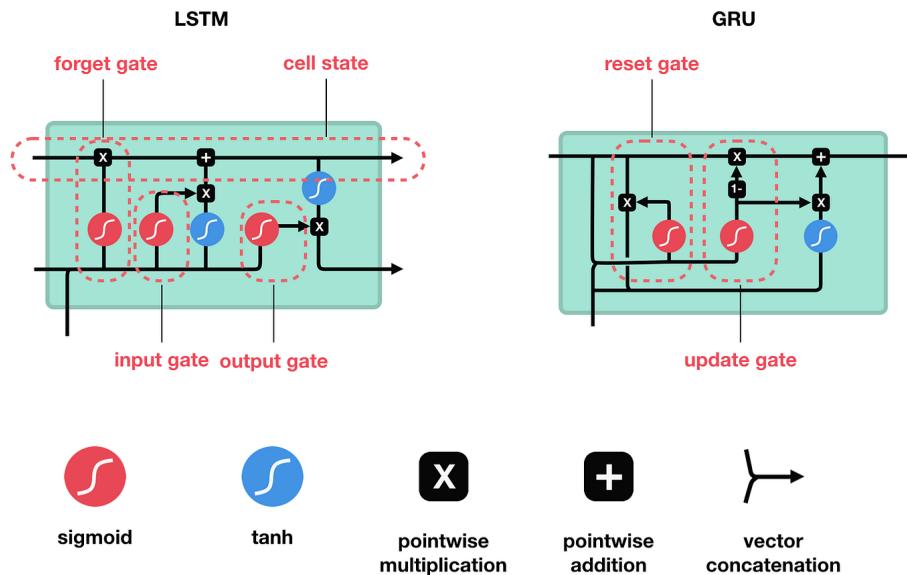
$$\tilde{a}^{(t)} = \tanh(W_h[r^{(t)} \odot a^{(t-1)}, x^{(t)}] + b_h) \quad (\text{candidate memory})$$

$$a^{(t)} = (1 - z^{(t)}) \odot a^{(t-1)} + z^{(t)} \odot \tilde{a}^{(t)} \quad (\text{final update})$$

Note

- Unlike LSTMs, GRUs do not use a separate cell state; the hidden state $a^{(t)}$ itself acts as memory. - GRUs are computationally lighter than LSTMs and often achieve comparable performance. - They are particularly useful when training data is limited or when faster training is needed.

The figure below shows the architecture of a GRU vs LSTM cell.



9 Bidirectional and Deep RNNs

9.1 Bidirectional RNNs (BRNNs)

Definition

A **Bidirectional RNN** processes a sequence in both directions:

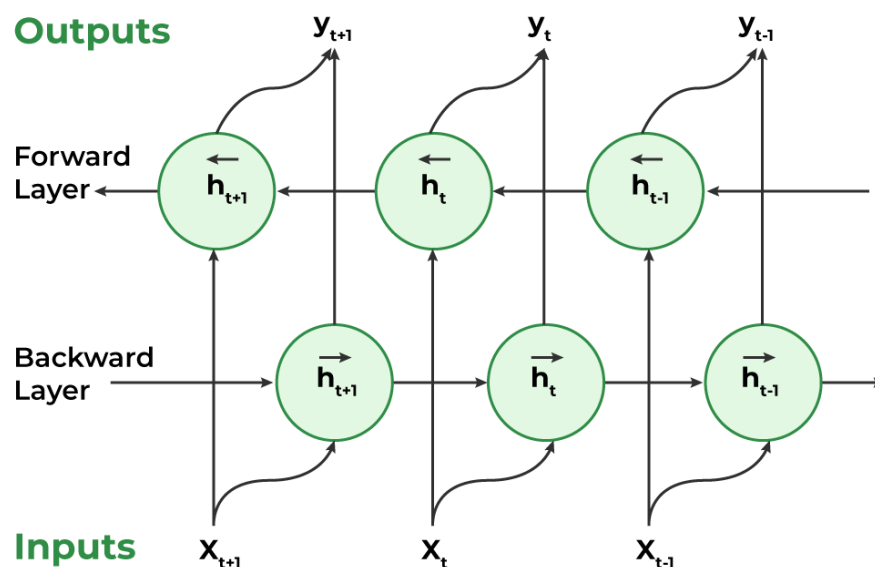
- A forward RNN reads from left to right: $h_{\text{fwd}}^{(t)} = \text{RNN}_{\text{fwd}}(x^{(t)}, h_{\text{fwd}}^{(t-1)})$.
- A backward RNN reads from right to left: $h_{\text{bwd}}^{(t)} = \text{RNN}_{\text{bwd}}(x^{(t)}, h_{\text{bwd}}^{(t+1)})$.

Their hidden states are combined (e.g., concatenated):

$$h^{(t)} = [h_{\text{fwd}}^{(t)}; h_{\text{bwd}}^{(t)}]$$

This architecture allows the model to access both **past** and **future** context at every time step.

The figure below illustrates the architecture of a Bidirectional RNN.



9.2 Deep RNNs

Definition

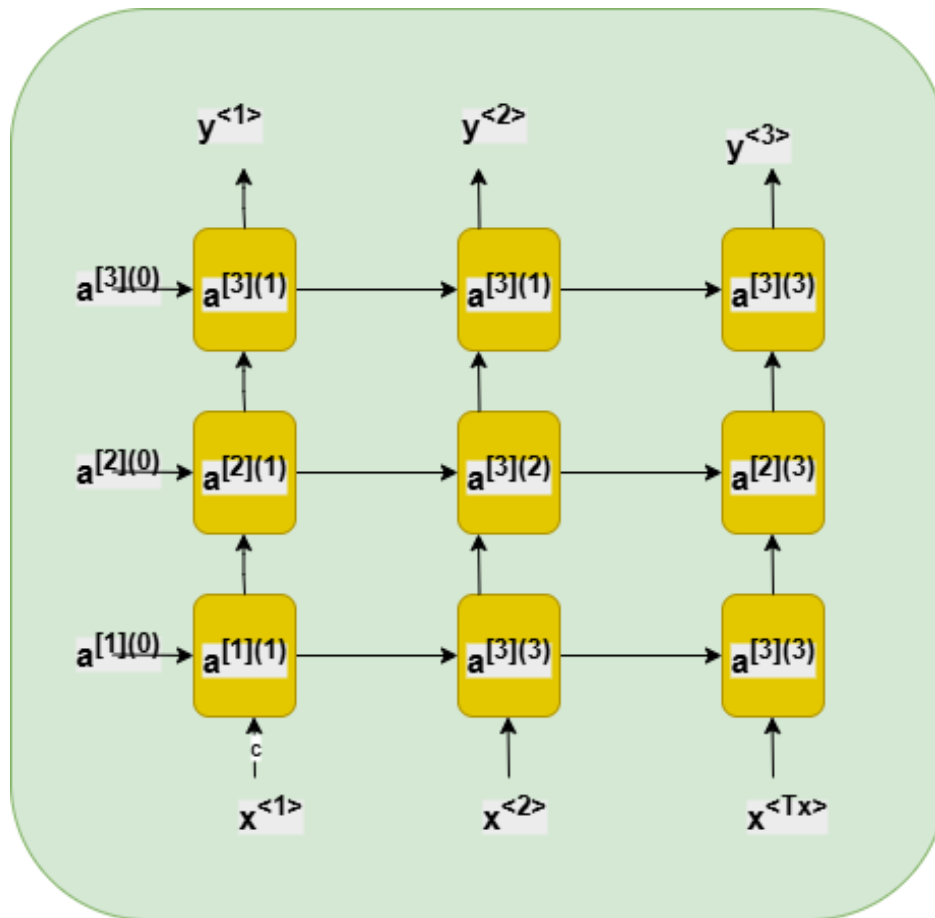
A **Deep RNN** stacks multiple RNN layers. The hidden state from one layer serves as input to the next:

$$a^{[l],\langle t \rangle} = g(W^{[l]}a^{[l-1],\langle t \rangle} + U^{[l]}a^{[l],\langle t-1 \rangle} + b^{[l]})$$

where $a^{[l-1],\langle t \rangle}$ is the hidden state of the previous layer.

This hierarchical structure enables the network to learn more complex temporal dependencies. In practice, LSTMs or GRUs are often stacked to form deep RNNs.

The figure below illustrates the architecture of a Deep RNN.



10 Word Embeddings

10.1 Introduction to Word Representations

Definition

Word embeddings are **dense vector representations of words** that capture semantic relationships. They allow algorithms to generalize across similar words, unlike one-hot vectors that treat each word as independent.

Computers cannot process letters or words directly; they require numeric representations. Early attempts to solve this included:

- **Unique IDs:** Assigning a number to each word; acts as a label but provides no semantic info.

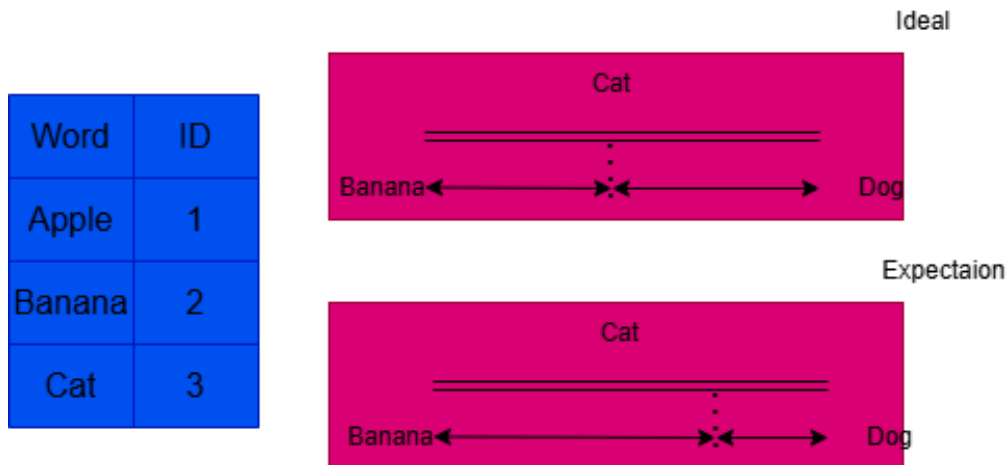


Figure 6: Using unique IDs fails because numerical labels create arbitrary distances between words that do not reflect their true semantic relationships.

- **One-hot Encoding:** Representing words as long binary vectors.

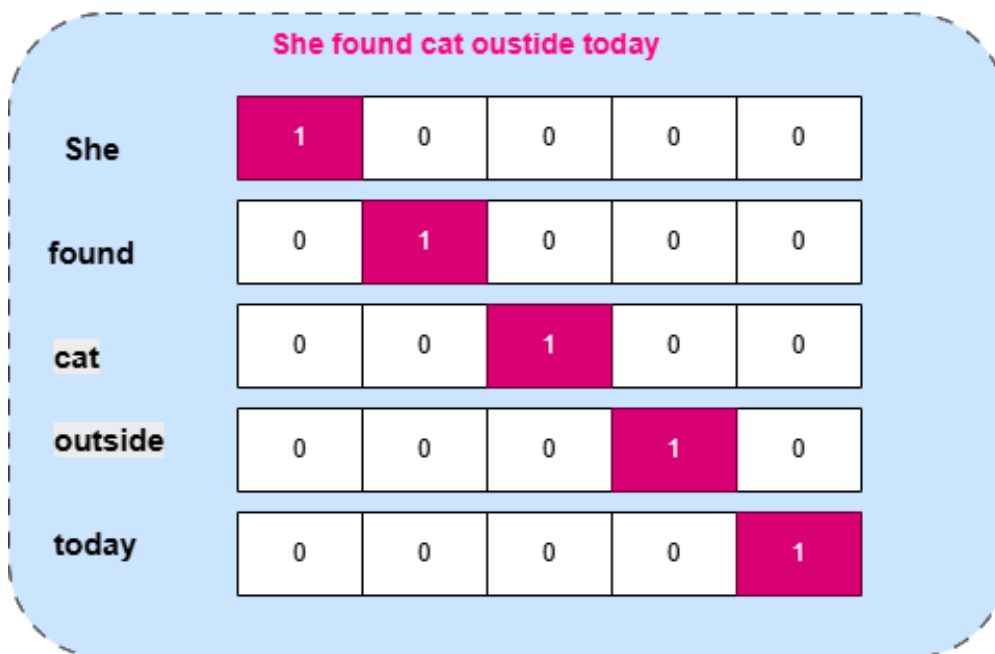


Figure 7: One-hot encoding: Sparse vectors where each word is an independent dimension.

- **Bag of Words (BoW):** Counting word frequency. While it captures presence, it ignores word order, meaning it cannot distinguish between sentences with the same words but different meanings.

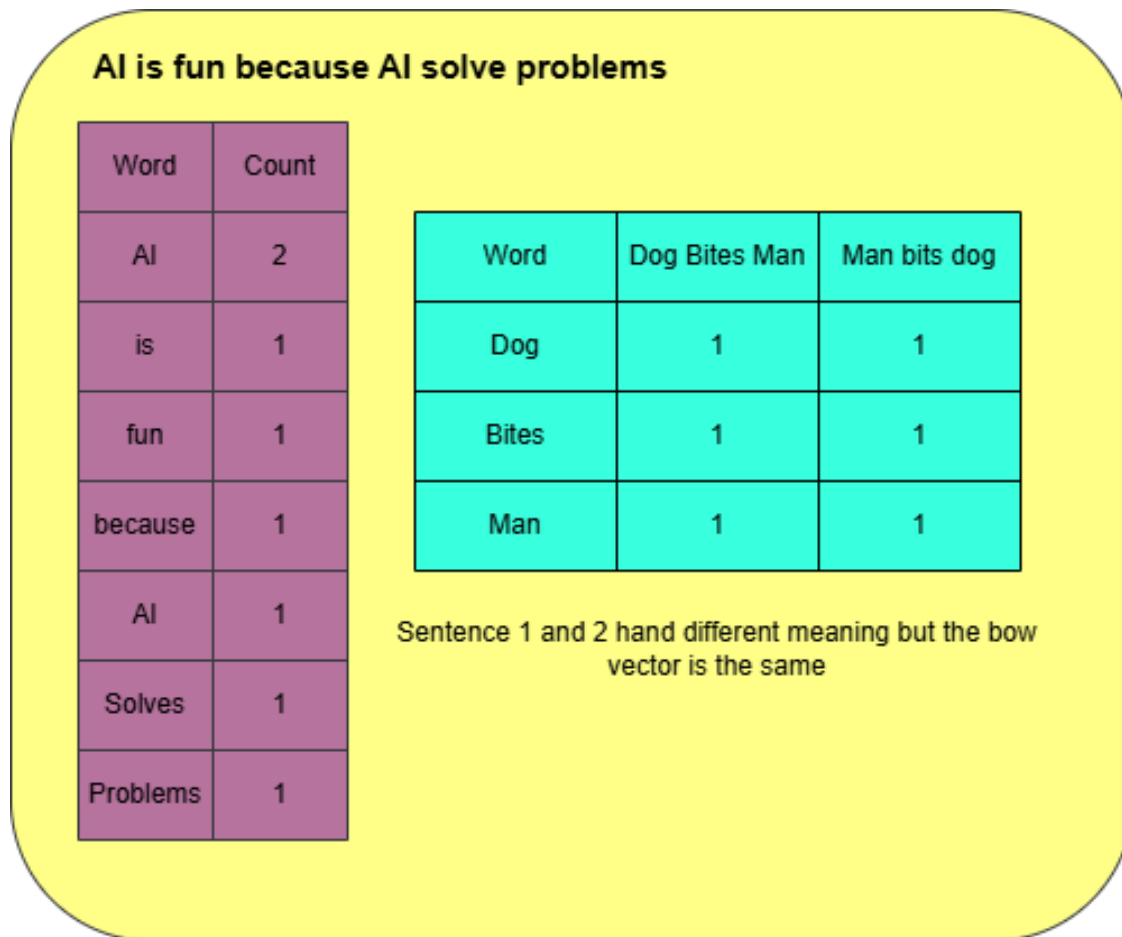


Figure 8: The limitation of Bag of Words: Different meanings result in identical vectors.

10.2 The Distributional Hypothesis

The core idea behind modern embeddings is that "you should know a word by the company it keeps." Words that appear in similar contexts are likely to have similar meanings.

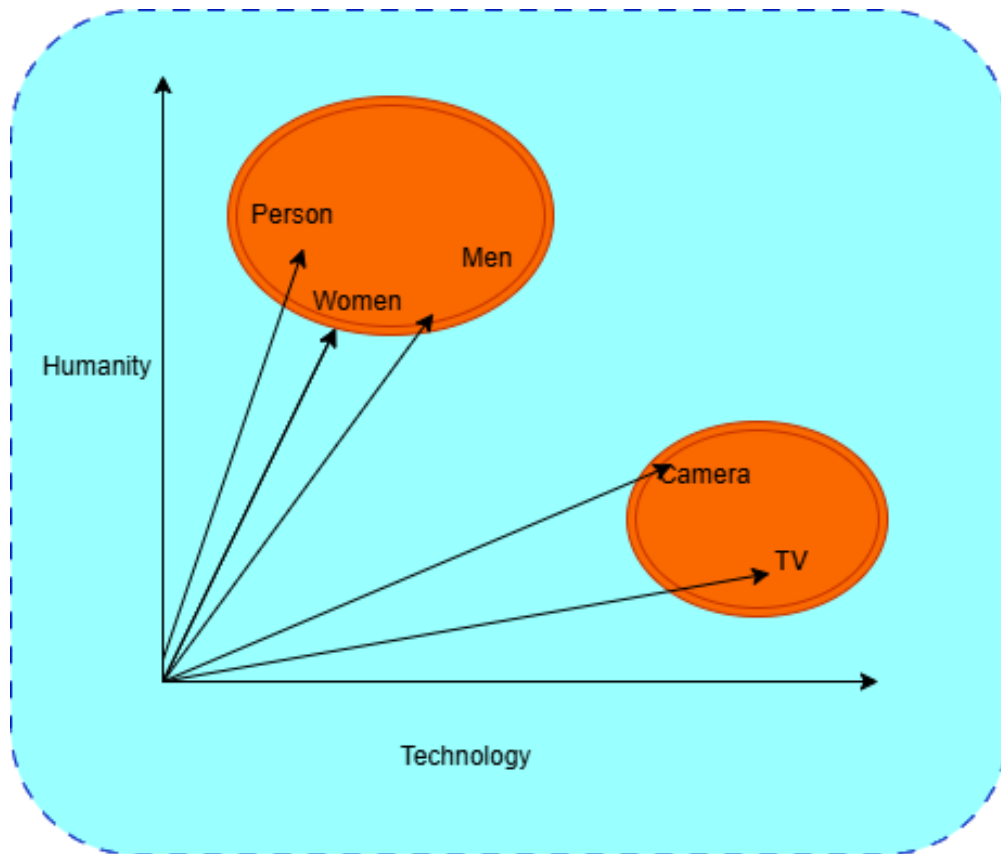


Figure 9: Learning via context: The computer deduces the meaning of "Glaf" based on surrounding actions like "barking."

10.3 Word2Vec Training Strategies

Google's 2013 Word2Vec [7] model popularized using neural networks to learn dense vectors via two primary strategies:

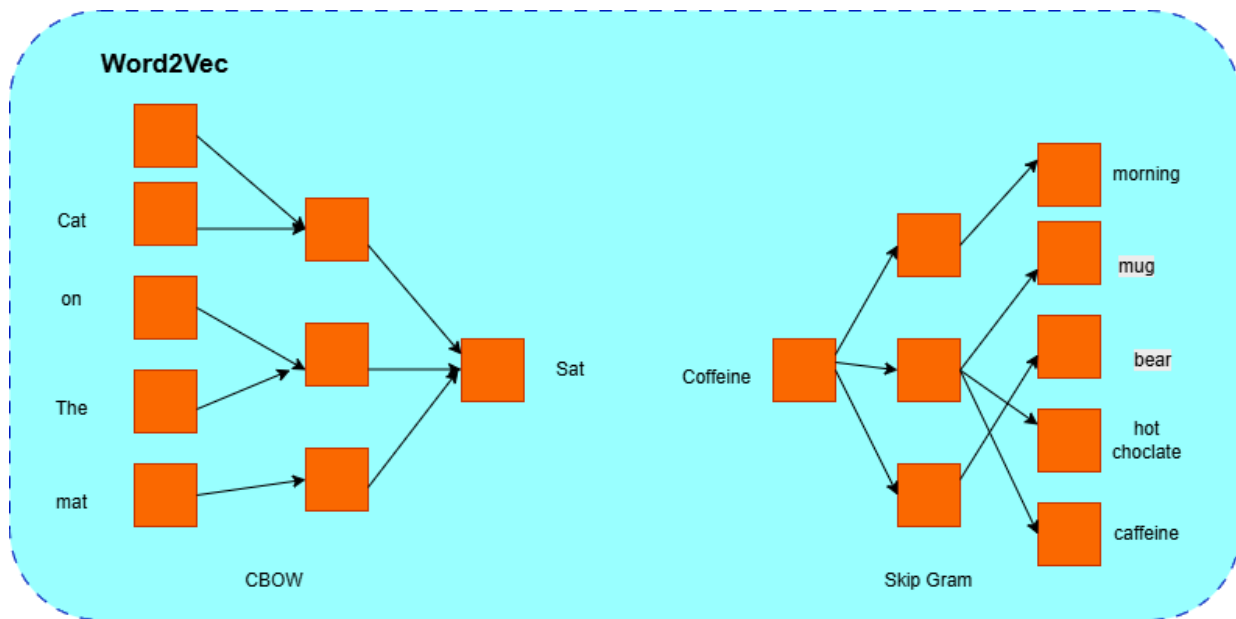


Figure 10: CBOW (predicting a word from context) vs. Skip-gram (predicting context from a word).

- **CBOW (Continuous Bag of Words):** Predicts a target word using the surrounding context.
- **Skip-gram:** Takes a single word and predicts its neighbors.

10.4 Interpretability and Latent Attributes

While dimensions in a vector space are not explicitly labeled, they often capture latent semantic "directions" like *Humanity* or *Technology*.

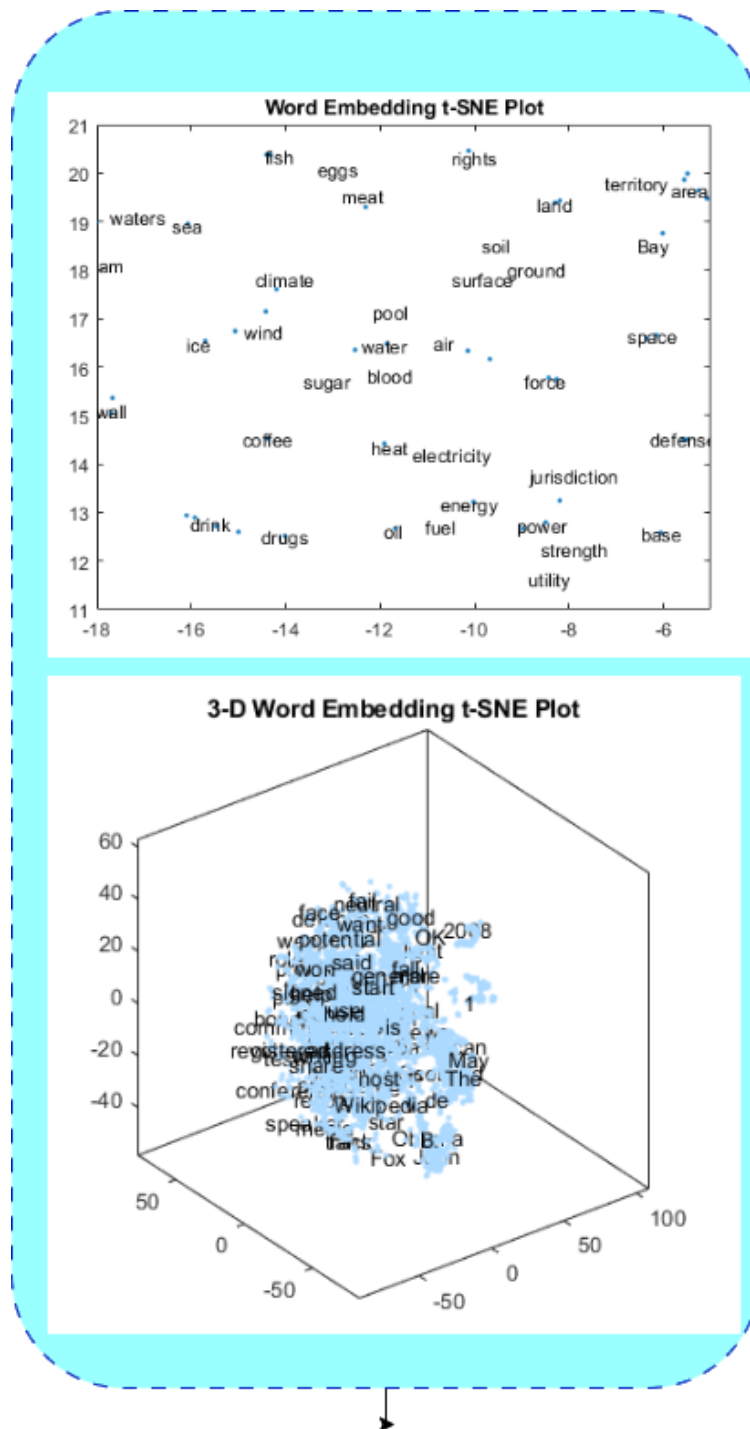


Figure 11: Words with similar latent attributes cluster together in the vector space.

10.5 Visualizing High-Dimensional Embeddings

To interpret embeddings (which often have 300+ dimensions), we use dimensionality reduction techniques like **t-SNE** to project them into 2D or 3D space.

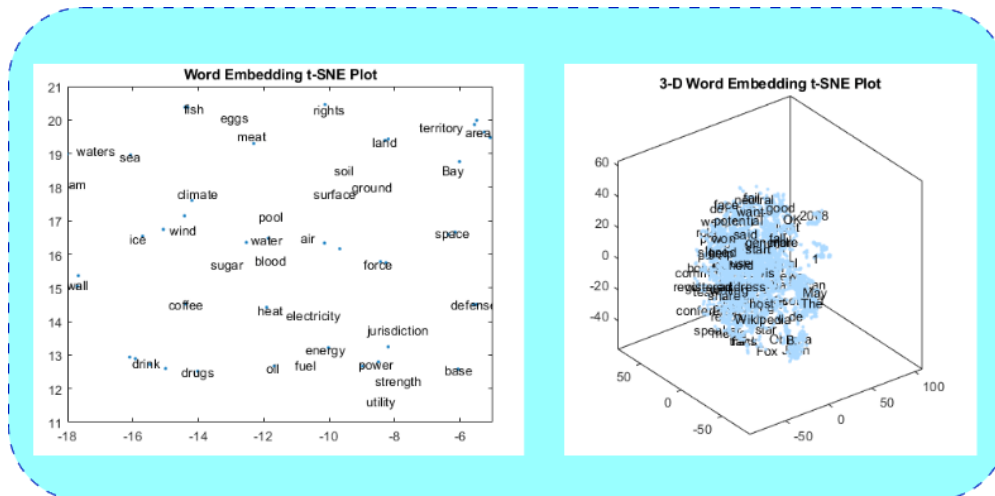


Figure 12: t-SNE visualizations of word embeddings in 2D and 3D space.

10.6 Static vs. Contextual Embeddings (Transformers)

A major limitation of early models like Word2Vec is that they produce **static vectors**. Modern Transformer-based models (like BERT and GPT) use **contextual embeddings**, generating a fresh vector for a word based on the specific sentence it is in.

11 Machine Translation

11.1 Conditional Language Models

Definition

A **machine translation model** is a conditional language model. Given a source sentence x , the model estimates the probability of a target sentence y :

$$P(y \mid x) = P(y_1, y_2, \dots, y_T \mid x)$$

Instead of initializing the decoder with zeros, the decoder is initialized with an **encoder representation** of the source sentence. This allows generating translations conditioned on the input sentence.

Example

French input: “*Jane visite l’Afrique en septembre*”

English output: “*Jane visits Africa in September*” The decoder uses the encoder’s representation of the French sentence to generate the English translation.

11.2 Greedy Search vs. Beam Search

Definition

Greedy Search: Pick the most probable word at each step. Simple, but may miss higher-probability full sentences.

Beam Search: Keep the top B sequences at each step, expanding each candidate and selecting the top B overall.

- Step 1: Evaluate probabilities for the first word and keep top B candidates.
- Step 2: Expand each candidate with all possible second words, compute joint probabilities, and keep top B .
- Repeat until the end-of-sentence symbol is generated.

Note

Beam search efficiently approximates the most probable translation without enumerating all possible sentences.

11.3 Refinements to Beam Search

Definition

Length Normalization: Probabilities of many words multiply to very small numbers. Use log probabilities:

$$\sum_{t=1}^T \log P(y_t \mid y_{<t}, x)$$

To avoid bias toward shorter sentences, normalize by length:

$$\frac{1}{T_y^\alpha} \sum_{t=1}^{T_y} \log P(y_t \mid y_{<t}, x)$$

where $\alpha \in [0, 1]$ is a tunable hyperparameter.

11.4 Error Analysis in Beam Search

Note

Beam search is approximate. Errors come from:

- The RNN (encoder-decoder) model.
- The beam search algorithm itself.

Given a reference translation y^* and a predicted translation $\hat{y}$:

1. Compute $P(y^*|x)$ and $P(\hat{y}|x)$.
2. If $P(y^*|x) > P(\hat{y}|x)$, the search algorithm caused the error.
3. If $P(y^*|x) \leq P(\hat{y}|x)$, the model itself is at fault.

11.5 BLEU Score

Definition

BLEU (Bilingual Evaluation Understudy) is an automatic metric for translation quality.

- Measures n-gram overlap with human reference translations.
- Outputs a score between 0 and 1 (or 0 and 100 if scaled).
- Useful for system comparison or hyperparameter tuning.

Example

French input: *“Le chat est sur le tapis”*

Reference English translation: *“The cat is on the mat”*

BLEU measures how closely a machine-generated translation matches the reference(s).

12 Attention Models: Intuition and Implementation

12.1 Motivation

Note

Traditional sequence-to-sequence models encode the entire input into a fixed-size hidden state before producing any output. While this works for short sequences, it struggles with longer or more complex inputs.

For example, consider the task of normalizing dates from natural text, such as:

“The Apollo Moon landing was on July 20, 1969.”

A standard encoder-decoder would need to remember the positions of the day, month, and year, which is difficult for long or irregular sentences. Attention models solve this by allowing the network to focus on only the relevant parts of the input for each output step.

12.2 Intuitive Example: Date Normalization

Example

Suppose the input sequence is:

‘‘July 20, 1969’’

The target output is:

1969-07-20

1. Step 1: Generate the year ‘‘1969’’

The decoder pays attention primarily to the token ‘‘1969’’ in the input. Attention weights $\alpha_{1,t'}$ indicate the importance of each input token for generating this first output.

$$c_1 = \sum_{t'=1}^{T_x} \alpha_{1,t'} a_{t'}$$

Here, $a_{t'}$ are the encoded features of each input token.

2. Step 2: Generate the month ‘‘07’’

Now the decoder focuses on ‘‘July,’’ with new attention weights $\alpha_{2,t'}$:

$$c_2 = \sum_{t'=1}^{T_x} \alpha_{2,t'} a_{t'}$$

3. Step 3: Generate the day ‘‘20’’

The decoder attends to ‘‘20’’ using $\alpha_{3,t'}$:

$$c_3 = \sum_{t'=1}^{T_x} \alpha_{3,t'} a_{t'}$$

At each step, the model focuses only on relevant input tokens, rather than memorizing the entire input sentence.

12.3 Formal Description of Attention

Let the input sequence be $X = (x_1, x_2, \dots, x_{T_x})$ and its encoded features by a bidirectional RNN be $a_{t'}$ for $t' = 1, \dots, T_x$.

At each output step t , we compute a context vector c_t as a weighted sum of the input features:

$$c_t = \sum_{t'=1}^{T_x} \alpha_{t,t'} a_{t'}$$

The attention weights $\alpha_{t,t'}$ satisfy:

$$\alpha_{t,t'} \geq 0, \quad \sum_{t'=1}^{T_x} \alpha_{t,t'} = 1$$

These weights indicate how much the decoder should focus on each input token when generating the t -th output.

12.4 Computing Attention Weights

We define a score $e_{t,t'}$ measuring the compatibility between the decoder state s_{t-1} and the input features $a_{t'}$:

$$e_{t,t'} = f_{\text{score}}(s_{t-1}, a_{t'})$$

The attention weights are obtained using a softmax over these scores:

$$\alpha_{t,t'} = \frac{\exp(e_{t,t'})}{\sum_{k=1}^{T_x} \exp(e_{t,k})}$$

Here, f_{score} can be a small neural network trained jointly with the sequence model. Intuitively, this network learns how much attention each input token deserves at each decoding step.

12.5 Decoding with Attention

The decoder RNN then generates each output y_t conditioned on the previous hidden state s_{t-1} , the previous output y_{t-1} , and the context vector c_t :

$$s_t = \text{RNN}(s_{t-1}, y_{t-1}, c_t), \quad y_t = \text{OutputLayer}(s_t)$$

This allows the network to generate outputs one step at a time, using only the relevant input information at each step, much like a human would focus on parts of a sentence when transcribing or translating.

12.6 Advantages and Applications

Key Advantages

- Improves performance on long sequences by avoiding the need to memorize the entire input.
- Provides interpretability: attention weights show which input tokens influence each output.
- Generalizes beyond machine translation: applicable to date normalization, speech recognition, and image captioning.

Example

Image Captioning: Given an image, an attention model can focus on different regions when generating each word in the caption, e.g., attending to the “dog” region when generating the word “dog”.

13 Numerical Example: Attention Mechanism

Consider a simple date normalization task:

Input: $x_1 = \text{"July"}, x_2 = \text{"20"}, x_3 = \text{"1969"} \rightarrow$ Output: $y_1 = \text{"1969"}, y_2 = \text{"07"}, y_3 = \text{"20"}$

Suppose each input token has a 2D feature vector:

$$a_1 = [1, 0], a_2 = [0, 1], a_3 = [1, 1], s_0 = [0.5, 0.5]$$

Step 1: Compute attention for y_1

$$e_{1,t'} = s_0 \cdot a_{t'} \implies e_1 = [0.5, 0.5, 1.0]$$

Softmax gives attention weights:

$$\alpha_1 = \text{softmax}(e_1) \approx [0.27, 0.27, 0.45]$$

Context vector:

$$c_1 = \sum \alpha_{1,t'} a_{t'} = [0.72, 0.72]$$

The decoder uses c_1 to output $y_1 = "1969"$.

Step 2: Compute attention for y_2

$$s_1 = \text{DecoderRNN}(s_0, y_1, c_1) = [0.6, 0.8]$$

$$e_2 = s_1 \cdot a_{t'} = [0.6, 0.8, 1.4]$$

$$\alpha_2 = \text{softmax}(e_2) \approx [0.21, 0.26, 0.53]$$

$$c_2 = \sum \alpha_{2,t'} a_{t'} = [0.74, 0.79]$$

Decoder outputs $y_2 = "07"$.

Step 3: Compute attention for y_3

$$s_2 = \text{DecoderRNN}(s_1, y_2, c_2) = [0.7, 0.9]$$

$$e_3 = s_2 \cdot a_{t'} = [0.7, 0.9, 1.6]$$

$$\alpha_3 = \text{softmax}(e_3) \approx [0.19, 0.23, 0.58]$$

$$c_3 = \sum \alpha_{3,t'} a_{t'} = [0.78, 0.81]$$

Decoder outputs $y_3 = "20"$.

Attention Table: Shows how focus shifts across input tokens for each output.

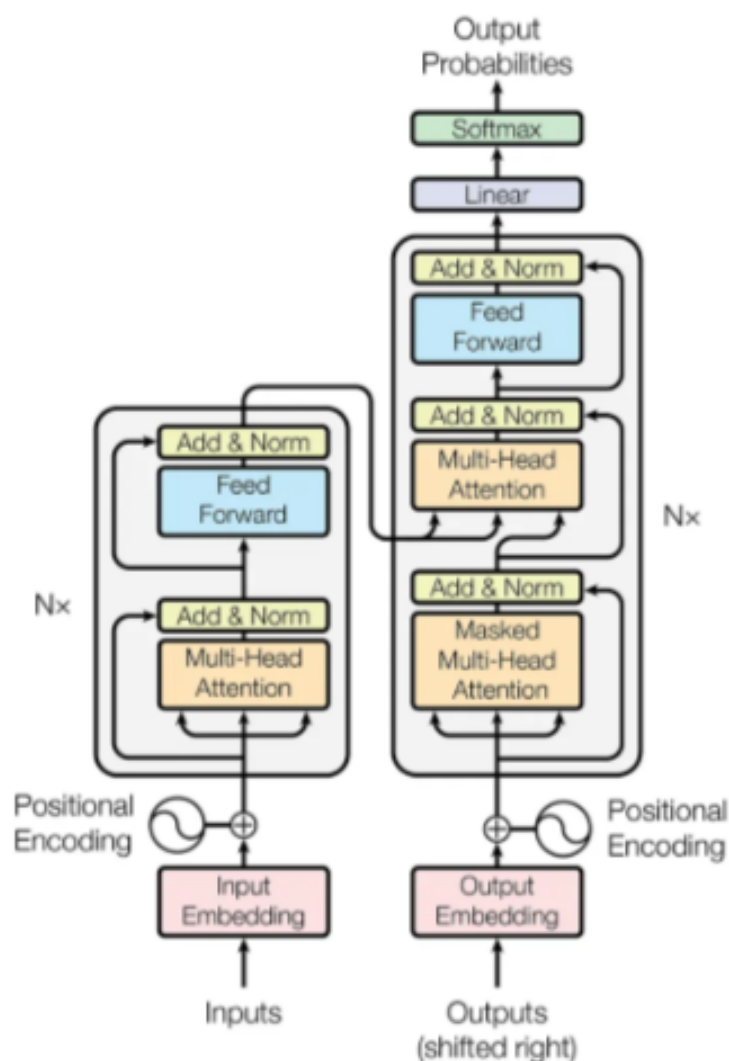
	x_1	x_2	x_3
y_1	0.27	0.27	0.45
y_2	0.21	0.26	0.53
y_3	0.19	0.23	0.58

Key idea: At each output step, the decoder focuses on the most relevant parts of the input, weighted by $\alpha_{t,t'}$, allowing correct sequence generation without memorizing the entire input.

14 Introduction to Transformers

So far, we have seen how attention allows a neural network to focus on relevant parts of the input while generating outputs. However, RNN-based models, even with attention, process sequences step by step. This sequential processing can be slow for long sequences and limits parallelization.

Transformers, introduced by Vaswani et al. in 2017 [9], solve this problem by relying entirely on attention mechanisms, removing recurrent operations. This allows the model to consider all positions in the sequence simultaneously, enabling efficient training and capturing long-range dependencies more effectively.



The Transformer — model architecture. (Credit: Vaswani et al, 2017)

Figure 13: The Transformer Architecture: A model based solely on attention mechanisms, bypassing recurrent layers for parallelized sequence processing.

14.1 Key Components of a Transformer

1. **Input Embeddings:** Each token in the sequence is converted into a dense vector representation. Positional encodings are added to retain the order of tokens.
2. **Self-Attention:** For every token, the model computes attention scores with all other tokens in the sequence, creating a weighted representation of the sequence context. This generalizes the attention idea from the previous RNN example but applies it in parallel to all tokens.
3. **Multi-Head Attention:** Instead of a single attention computation, the model uses multiple "heads," each learning different types of relationships between tokens. The outputs of these heads are concatenated and linearly transformed.
4. **Feed-Forward Layers:** After attention, each token's representation is passed through a fully connected feed-forward network, applied identically to all tokens.
5. **Residual Connections and Layer Normalization:** To stabilize training and improve gradient flow, residual connections and normalization are applied after each sub-layer.
6. **Stacked Encoder and Decoder Layers:** The Transformer consists of an encoder that processes the input sequence and a decoder that generates the output sequence. Each is composed of several identical layers with attention and feed-forward networks.

14.2 Self-Attention Intuition

Consider a sentence: "*The cat sat on the mat*". In self-attention, the word "cat" computes attention scores with every word in the sentence to decide which words are important to represent "cat" in context. For example, "sat" and "mat" may receive higher attention scores, while "the" may receive a lower score.

14.3 Advantages of Transformers

- **Parallelization:** Unlike RNNs, Transformers process all tokens simultaneously, drastically speeding up training.
- **Long-Range Dependencies:** Each token can directly attend to any other token, making it easier to capture long-term dependencies.
- **Scalability:** Transformers scale well with data and model size, which has enabled models like BERT, GPT, and other large language models.

14.4 From Attention to Transformers

We can think of a Transformer as a natural extension of the attention mechanism we studied:

- **RNN + Attention:** Processes one token at a time, computes context via attention.
- **Transformer:** Processes all tokens simultaneously, uses self-attention and multi-head attention to capture all relationships efficiently.

In the next section, we will illustrate a simple Transformer encoder block with a small sequence to make this intuition concrete.

15 Large Language Models (LLMs)

Large Language Models (LLMs) are neural networks trained on massive amounts of text to understand and generate human-like language. They are typically based on the Transformer architecture, which allows them to capture complex relationships between words across long contexts.

15.1 Key Characteristics of LLMs

- **Scale:** LLMs have billions of parameters (weights), enabling them to store vast amounts of linguistic knowledge.
- **Context Understanding:** They can capture the meaning of words based on surrounding context, thanks to the self-attention mechanism.
- **Text Generation:** LLMs can generate coherent and contextually relevant text, translate languages, summarize documents, answer questions, and more.
- **Pretraining and Fine-Tuning:** LLMs are first pretrained on a large general text corpus and can then be fine-tuned for specific tasks like translation, summarization, or code generation.

15.2 Examples of LLMs

Some well-known LLMs include:

- **GPT (Generative Pretrained Transformer):** Focused on generating coherent text.

- **BERT (Bidirectional Encoder Representations from Transformers):** Focused on understanding text, commonly used for question answering and classification.
- **T5 (Text-to-Text Transfer Transformer):** Treats all tasks as text-to-text problems.
- **PaLM, LLaMA, Mistral:** Large models designed for research or general-purpose natural language understanding and generation.

15.3 Why LLMs Work Well

LLMs leverage the Transformer's ability to:

- Capture long-range dependencies between words in a sequence.
- Learn rich, contextual word embeddings.
- Scale with more parameters and more training data to improve performance.

15.4 Applications of LLMs

- Conversational AI and chatbots.
- Text summarization and translation.
- Code generation and assistance.
- Question answering and information retrieval.
- Creative writing and content generation.

15.5 Inside CHAT GPT 2

Note

The following section is adapted from the *Transformer Explainer* by the Polo Club of Data Science [10]. Every generative Transformer operates through three primary stages: Embeddings, Transformer Blocks, and Output Probabilities.

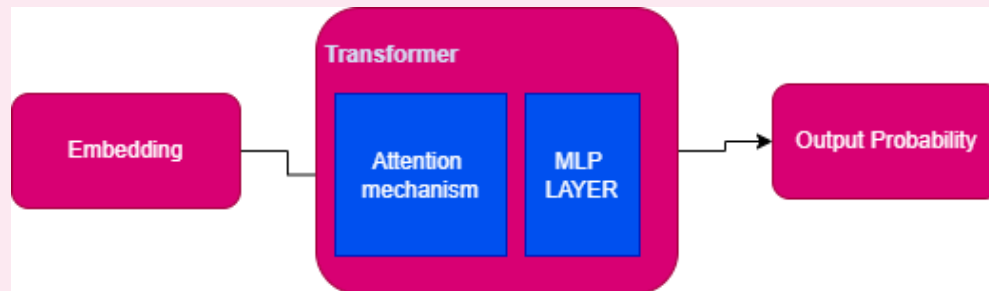


Figure 14: High-level overview of the Transformer pipeline: from input tokens to predicted probabilities.

15.5.1 1. The Embedding Layer: Mapping Text to Vector Space

Computers process numerical values, not raw text. The transition from a prompt (e.g., "Data visualization empowers users to") to a model-ready format involves four steps:

- **Tokenization:** The text is broken into smaller units called tokens. For example, GPT-2 uses a vocabulary of 50,257 unique tokens.
- **Token Embedding:** Each token ID is mapped to a high-dimensional vector (768 dimensions in GPT-2 small). This captures semantic meaning by placing similar tokens closer together in vector space.
- **Positional Encoding:** Since Transformers process all tokens simultaneously, they lack an inherent sense of order. Positional vectors are added to indicate where each token sits in the sequence.
- **Final Embedding:** The sum of token and positional encodings forms the final representation.

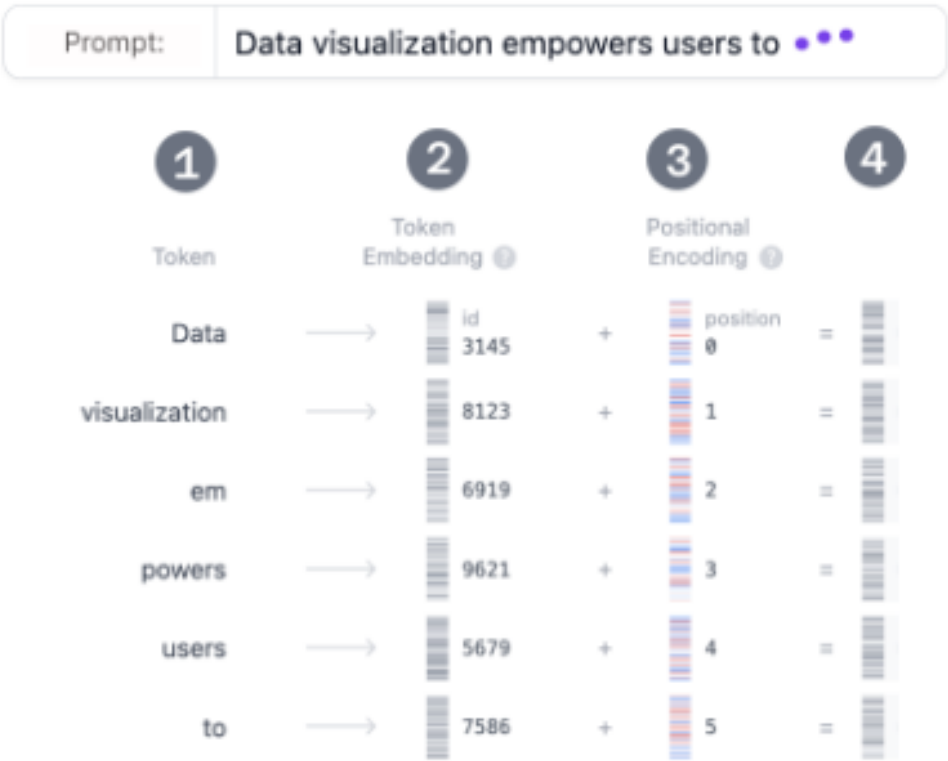


Figure 15: The transformation of raw text into final embeddings via tokenization and positional encoding.

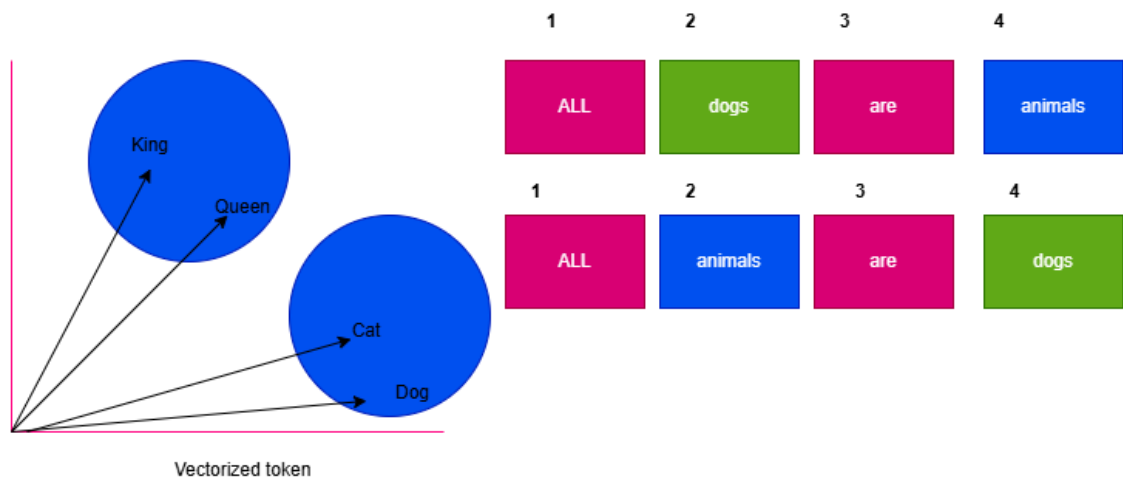


Figure 16: Positional Encoding: Distinguishing between sentences with identical words but different meanings by injecting sequence order into the vector representation.

As shown in the figure, different models use various methods for positional encoding. GPT-2 trains its own positional encoding matrix from scratch, which is then summed with the

token embeddings to ensure the model understands that "dogs" at position 2 is different from "dogs" at position 4.

15.5.2 The Transformer Block: The Engine of Inference

The core processing occurs within stacked Transformer blocks (GPT-2 small uses 12). Each block contains two main sub-layers:

Multi-Head Self-Attention Step 1: Query, Key, and Value Matrices This mechanism allows tokens to "communicate" with each other to gain context. It utilizes a web-search analogy:

- **Query (Q):** What the token is looking for.
- **Key (K):** What information other tokens offer.
- **Value (V):** The actual information content.

The model computes a dot product between Q and K , applies a **Mask** (to prevent "peeking" at future tokens), and uses a Softmax function to determine which V values are most relevant.

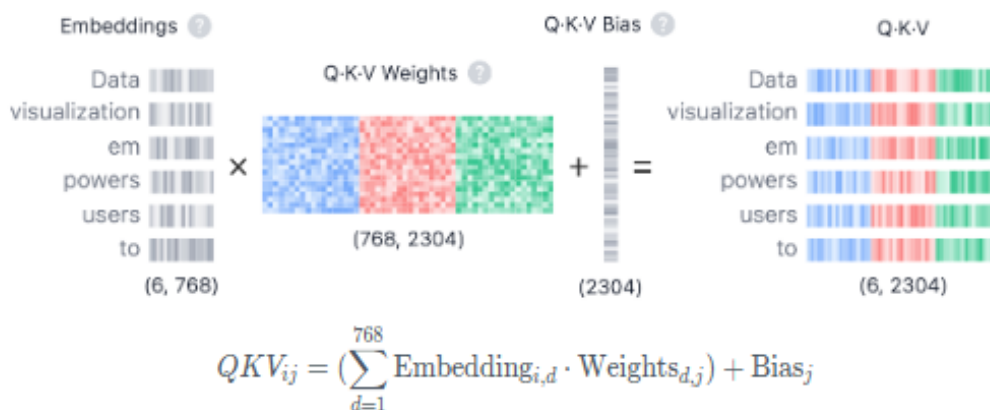


Figure 2. Computing Query, Key, and Value matrices from the original embedding.

Figure 17: The QKV Transformation: Mapping 768-dimensional embeddings into a higher-dimensional space to prepare for multi-head splitting.

As shown in Figure 2, this process projects the semantic data into a format where tokens can "search" for one another. In GPT-2 (small), the resulting 2304-dimensional vector is then split into 12 independent heads to capture diverse linguistic relationships simultaneously.

Step 2: Multi-Head Splitting The Query, Key, and Value vectors are subdivided into multiple independent "heads"—specifically 12 heads in the case of GPT-2 (small). Each head processes a unique segment of the embedding space, allowing the model to simultaneously attend to different types of information, such as grammatical structure (syntax) and thematic meaning (semantics). This parallel architecture significantly enhances the model's representational power.

Step 3: Masked Self-Attention Each head performs a masked self-attention calculation to determine which parts of the input sequence are most relevant. This process ensures the model only "looks" at previous tokens when predicting the next one, preventing it from accessing future information.

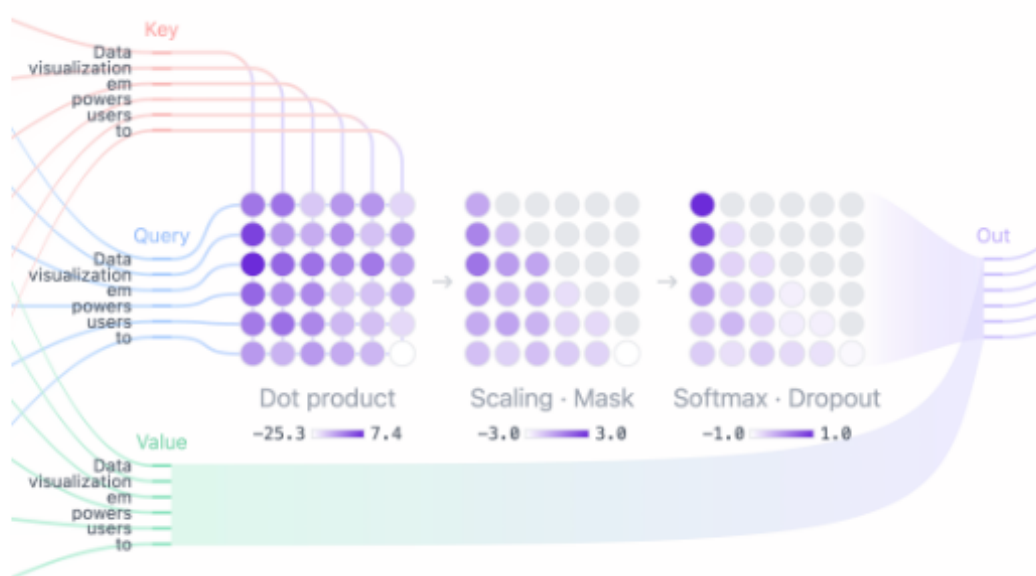


Figure 18: The Masked Self-Attention Workflow: From dot product scores to a causal probability distribution.

The calculation follows these critical stages as shown in Figure 3:

- **Dot Product:** The Q and K matrices are multiplied to produce an attention score matrix, reflecting the relationship between all tokens in the sequence.
- **Scaling and Masking:** Scores are scaled for stability, and a mask is applied to the upper triangle of the matrix. By setting future token scores to negative infinity ($-\infty$), the model is forced to ignore them during the next step.
- **Softmax and Dropout:** The masked scores are converted into probabilities via a Softmax function, ensuring each row sums to one. This creates a clear map of which previous tokens are most relevant to the current prediction.

Step 4: Output and Concatenation After calculating the masked self-attention scores, the model multiplies these probabilities with the **Value (V)** matrix to produce the final output for each head.

Since GPT-2 utilizes 12 distinct self-attention heads, each provides a different "perspective" or relationship mapping for the tokens. To consolidate this information:

- **Concatenation:** The individual outputs from all 12 heads are joined back together into a single, comprehensive vector.
- **Linear Projection:** This concatenated result is passed through a final linear layer to project it back into the model's standard hidden dimension (768 for GPT-2 small), preparing it for the next stage of the block.

Note

By combining the "opinions" of all 12 heads, the model ensures that the resulting token representation captures a rich mix of syntactic and semantic context simultaneously.

MLP (Multilayer Perceptron) While attention routes information *between* tokens, the MLP layer refines each token *individually*. It projects the 768-dim vector into a 3072-dim space to capture complex patterns before compressing it back.

15.5.3 Output Probabilities

The final representation is passed through a linear layer and a **Softmax** operation. This converts internal vectors into a probability distribution across the entire vocabulary, allowing the model to sample the most likely next token.

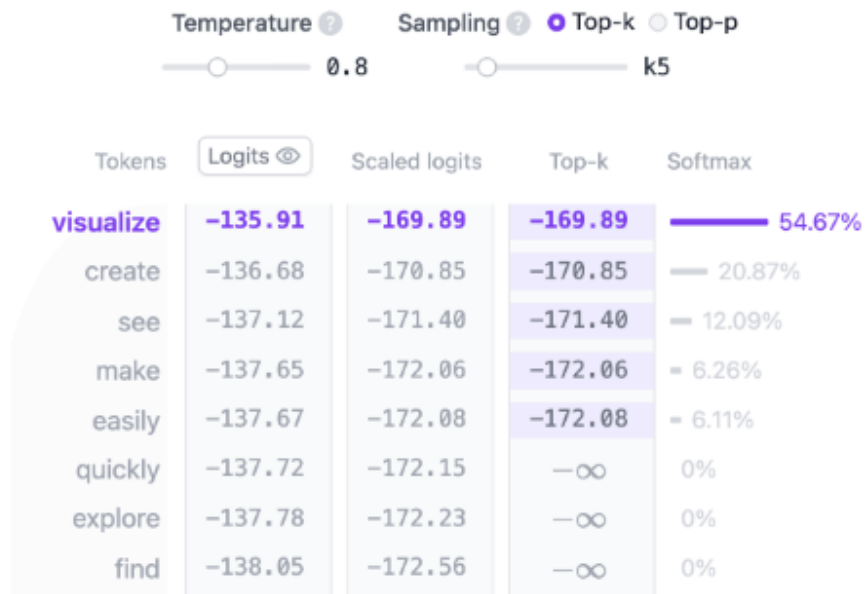


Figure 19: From Logits to Probabilities: The final stage of the Transformer where internal representations are decoded to rank and sample the next token.

In short, LLMs are powerful tools built on Transformers that can perform a wide variety of natural language tasks by leveraging massive amounts of learned linguistic knowledge.

References

- [1] A. Markov (1906), *Extension of the Limit Theorems of Probability Theory to a Sum of Variables Connected in a Chain*
- [2] J.J. Hopfield (1982), *Neural networks and physical systems with emergent collective computational abilities*
- [3] D.E. Rumelhart, G.E. Hinton, R.J. Williams (1986), *Learning representations by back-propagating errors*
- [4] S. Hochreiter, J. Schmidhuber (1997), *Long Short-Term Memory*
- [5] K. Cho et al. (2014), *Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation*
- [6] J. Schmidhuber (2015), *Deep Learning in Neural Networks: An Overview (survey with historical perspective)*
- [7] T. Mikolov et al. (2013), *Efficient Estimation of Word Representations in Vector Space*, [arXiv:1301.3781](#)
- [8] D. Bahdanau, K. Cho, Y. Bengio (2014), *Neural Machine Translation by Jointly Learning to Align and Translate*, [arXiv:1409.0473](#)
- [9] A. Vaswani et al. (2017), *Attention Is All You Need*, [arXiv:1706.03762](#)
- [10] A. Wang et al. (2024), *Transformer Explainer: LLM Transformer Model Visually Explained*, [Polo Club of Data Science](#), [Georgia Tech](#)